

POLITECNICO
MILANO 1863

HOP-PAAL: A Formal Frogger Journey

Formal Methods for Software Engineering
A.Y. 2025-2026

Daniele Crippa, Dario Cambi and Giorgio Ciraci

Politecnico di Milano
daniele5.crippa@mail.polimi.it
dario.cambi@mail.polimi.it
giorgio.ciraci@mail.polimi.it

Abstract

UPPAAL is a tool for modeling, simulating, and verifying real-time systems as networks of Timed Automata. In this report, we present the modeling and formal verification of a simplified version of Frogger, a classic arcade game. Both a vanilla and a stochastic version of the model are developed and analyzed.

1 Introduction

Frogger is a classic arcade video game released in 1981, one of the most iconic titles of its era. The primary goal of the game is to successfully guide a frog into a **home bay** as many times as possible, requiring the player to cross a **busy highway** and a **dangerous river**, avoiding moving cars and jumping on logs and turtles to avoid falling into the water.

Formal verification is a technique used to **prove or disprove the correctness of a system** with respect to a formal specification. While commonly applied to **hardware and safety-critical software**, it can also be used to model and verify the behavior of more unconventional systems, such as video games, which are rarely formally defined.

UPPAAL is a tool designed for **modeling, simulating, and verifying real-time systems** using networks of **Timed Automata**. It provides a graphical interface alongside a powerful **query language** that allows the user to formally check properties on the system.

In this project, UPPAAL is applied to **model and formally verify** a simplified version of Frogger, covering all its **core mechanics** as defined in the specifications. The goal is to demonstrate that the model **behaves correctly under all possible conditions**, using both a **vanilla** and a **stochastic** version of the automata network.

1.1 Assumptions and further clarifications

The following assumptions are made on the specifications:

- One time unit in UPPAAL corresponds to 0.1 seconds
- Between any two consecutive moves of Frogger, at least 1 time unit must elapse; this prevents unrealistic Zeno behaviors
- The game timer is assumed to start when Frogger respawns, ignoring the respawn cooldown. As a consequence, the last playable run starts with a game timer of 2 seconds, during which Frogger is bound to die, since the respawn cooldown and the game timer are both equal to 2 seconds.
- The initial level is zero.

After further context clarifications with the professor we also defined the following edge cases or ambiguous situations:

-

2 Model Description

The model is based on 4 primary components:

- *Frogger*
- *Vehicle Lane*
- *Floating Lane*
- *Diving Turtles Manager*

Frogger

The protagonist of the game, it starts with 5 lives and must reach all 5 home bays to complete a level. The frog dies upon collision with a vehicle, contact with open water, or expiration of the time limit.

Vehicle Lane

Models a road lane populated by moving vehicles of varying speed and size. Each lane defines a fixed direction of travel, and a collision with the frog results in the loss of a life.

Floating Lane

Models a river lane composed of floating objects, such as logs and turtles. When the frog stands on a floating object, it is carried along with it. Landing on an empty water cell results in immediate death.

Diving Turtles Manager

Manages the submersion cycle of diving turtles, cycling through the states *Emerged*, *Diving*, *Submerged*, and *Emerging*. When turtles submerge, a broadcast signal notifies the Floating Lane, which treats the occupied cell as open water, causing the frog to die if standing on it.

2.1 Overall Design Choices

Before describing each template in detail, we summarise the main design choices that shape the whole model and that are shared across its components.

Game Map

The game map is encoded as a fixed matrix of size `GRID_H`×`LANE_W`, where each cell holds its initial logical content. A central choice is that moving objects are *never* physically rewritten in this matrix: their motion is represented by a single per-lane shift variable inside the lane templates. This keeps the representation compact and avoids continuously updating a large array, which would otherwise enlarge the state space.

Lane Abstraction

Roads and rivers are both modelled as lanes, since they share most of their logic; this avoids modelling each element separately and prevents the state space from exploding. The motion of objects is not rewritten in the map matrix but represented by a single per-lane shift variable, keeping the representation compact.

The logical lane width `LANE_W` is intentionally larger than the visible grid `GRID_W` (20 against 18): the two extra off-screen cells model the re-entry delay required by the specification, so that an object leaving the map does not reappear immediately on the other side. For the same reason, `frog_x` ranges over the full `LANE_W`, allowing Frogger to be carried slightly off-screen while riding a floating object.

Lanes Handle Frogger's Death

The decision to kill Frogger is delegated to the lanes rather than to the frog itself. Each lane knows the global position of Frogger, so it is the natural place to detect a collision or an unsafe cell; when this happens, it requests the death through the `kill_frog` channel, which **Frogger** automaton consumes by removing a life and respawning. This keeps the collision logic local to each lane and leaves the frog simple and focused on its own movement and life management.

Turtles Management

The global submersion cycle of the diving turtles is delegated to a separate template, so that the shared timing is defined in one place and simply observed by the lanes.

Moreover, the water gaps between turtles of the same group are encoded as turtle cells, since Frogger standing on those cells is equally safe, the two are semantically equivalent.

2.2 Frogger

The frogger template contains the main logic of the frog, including its movement, life management and points assignment.

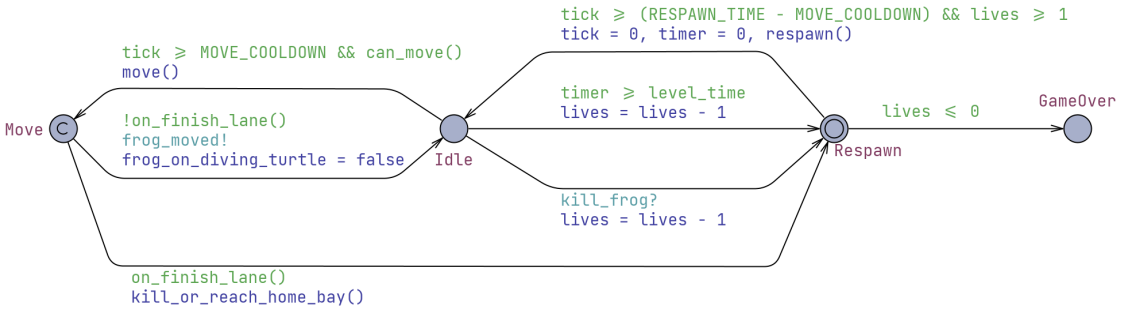


Figure 1: Frogger Template

The design was made around its possible states:

1. **Idle** is the central state where the frog is waiting between actions. Three transitions can fire from here: if the move is legal and the cooldown time is over, the frog can go to the **Move** state; it can be killed either by another template through the `kill_frog` channel or by running out of time, losing a life and sending it to the **Respawn** state.

2. **Move** is a *committed* state in which Frogger enters after a movement command; here, immediately is determined the outcome of its move. If it lands on a finishing lane, it resolves whether a home bay is reached through; otherwise it simply update the position of Frogger and returns to **Idle**.

3. **Respawn** is reached whenever the frog loses a life or reaches an home bay. From here, if at least one life remains, the frog waits for the respawn timing and then returns to **Idle**. If `lives <= 0`, it transitions to **GameOver** instead.

4. **GameOver**: the final state of the automaton, entered when the frog has no lives left.

2.3 Vehicle Lane

The vehicle lane template models a single lane of the road, populated by vehicles that move at the same speed and share the same size; both parameters are constants and defined at the instantiation of the template.

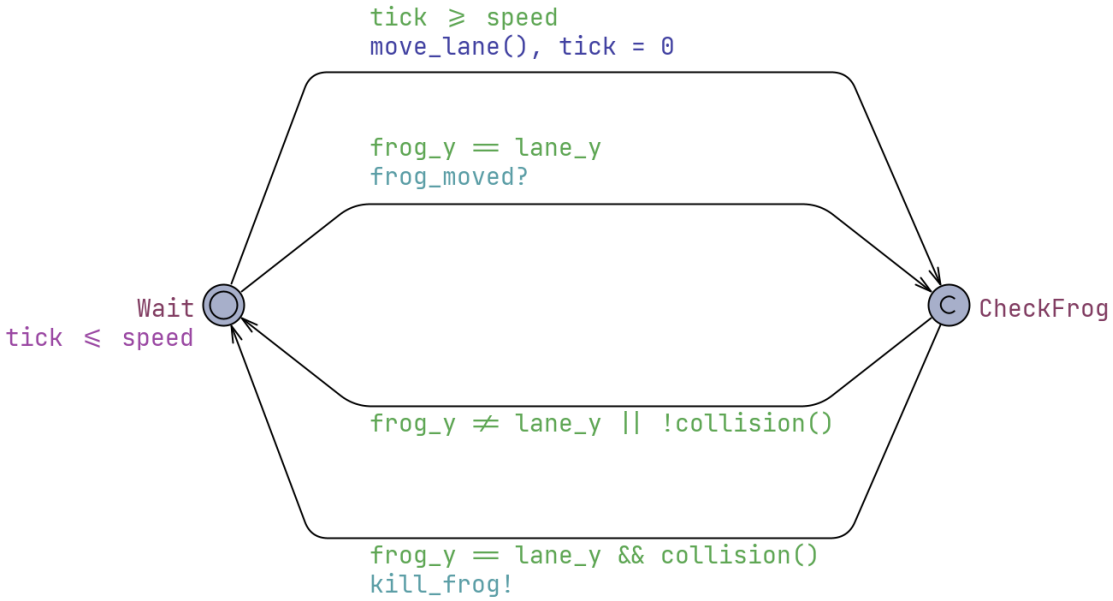


Figure 2: Vehicle Lane Template

The template is realised through two states, **Wait** and **CheckFrog**. In **Wait**, the lane lets time pass until the interal clock reaches the speed; when this happens, it shift the vehicles with the function `move_lane()`, resets the timer, and transitions to **CheckFrog**. This is a *committed* state, so the collision check is performed instantaneously: if a vehicle occupies Frogger's position, it signals the death through the `kill_frog` channel and returns to **Wait**; otherwise it returns to **Wait** without sending any signal.

A second path into **CheckFrog** is triggered when the frog moves: the lane is notified through the `frog_moved` channel, allowing it to verify immediately whether the frog's new position is occupied by a vehicle; if so it signals it through the channel `kill_frog`.

2.4 Floating Object Lane

The floating lane template models a river lane, where the frog can only survive by standing on floating objects, such as logs and turtles.

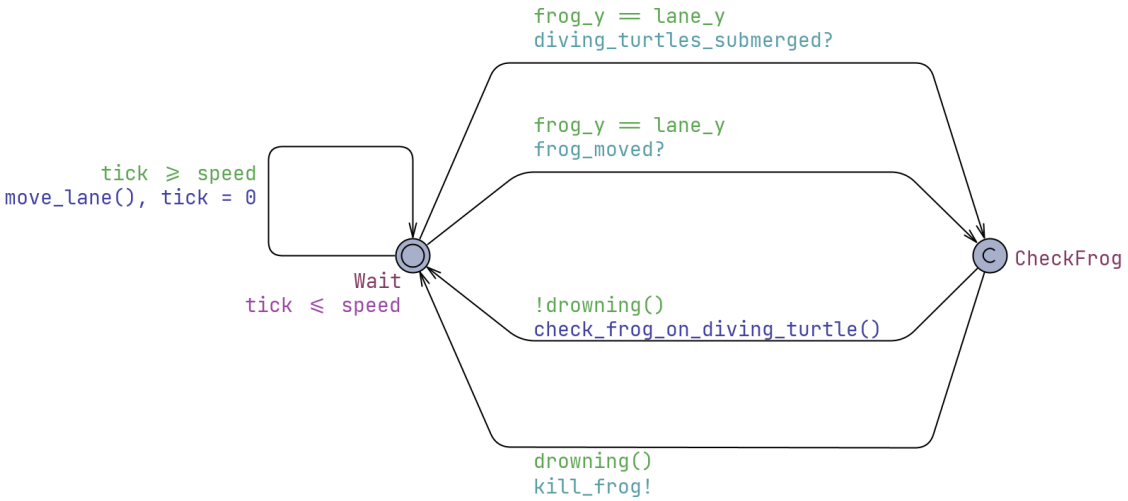


Figure 3: Floating Object Lane Template

The template is structured in two main states, **Wait** and **CheckFrog**. In **Wait**, the lane lets time pass until the internal clock reaches the speed of the lane; when this happens, it shifts

the floating objects with the function `move_lane()` and transitions to **CheckFrog**. If the frog is currently standing on a floating object, its global position is updated within the same function, so that it is carried along with the object.

CheckFrog is a *committed* state where the lane verifies whether the frog is safe, and it can be reached through three different paths. The first is the periodic movement just described: after shifting, the lane checks if the frog is drowning with the function `drowning()`, and if it not, it safely returns to **Wait**; otherwise it signals the death through the `kill_frog` channel. The second path is triggered when the frog moves onto the lane, notified through the `frog_moved` channel, allowing the lane to immediately check whether the new position is occupied by a floating object. The third is triggered when the diving turtles submerge, signalled through the `diving_turtles_submerged` channel: if the frog was standing on those turtles, the cell becomes open water and the frog drowns.

2.5 Diving Turtles Manager

The diving turtles manager template models the periodic submersion cycle of the diving turtles through four states: *Emerged*, *Diving*, *Submerged*, and *Emerging*.

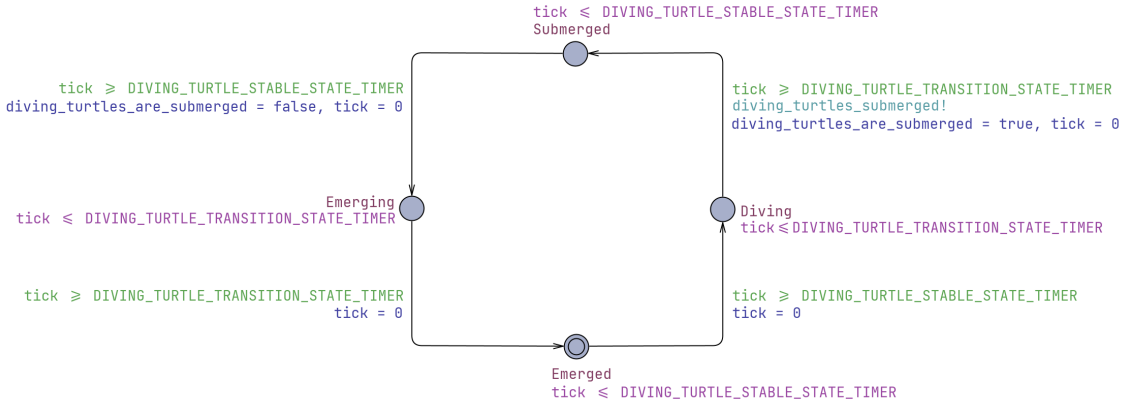


Figure 4: Diving Turtles Manager Template

2.6 Channels

The communication between templates relies on three synchronisation channels, each chosen as broadcast or binary according to how many automata must react to the event:

- `kill_frog` (binary): requests the death of Frogger. It is sent by a **VehicleLane** when a vehicle overlaps the frog, or by a **FloatingLane** when the frog is on unsafe water or on a submerged diving-turtle group. It is received only by the **Frogger** automaton, which removes one life and respawns the frog.

- `frog_moved` (broadcast): announces that Frogger has completed a movement step. It is received by the **VehicleLane** and **FloatingLane** templates, so that the current lane where Frogger is standing can check whether the new position is unsafe; if so it signals the death through the `kill_frog` channel. It is broadcast because a priori is unkown which lane is interested in the message.

- `diving_turtles_submerged` (broadcast): announces that the diving turtles have entered the submerged phase. It is sent by the **Diving Turtle Manager** and received by the **FloatingLane** templates containing diving turtles, which then check whether Frogger is standing on a submerged turtle.

2.7 Stochastic Variation

3 Properties Verification

In this section are shown the proprieties that have been checked over the system to be sure that it behaves as expected and the queries requested are verified. The properties we discuss here can be found as queries in the Verifier section of UPPAAL. Details about the time and RAM usage of the verification process can be found in the report.

3.1 System verification

A home bay is reachable

$$\text{(III)}: \quad \exists\Diamond \left(frog_y = 12 \ \wedge \ home_bays[frog_x] = 1\right)$$

It's saying that exists at least one path that eventually will lead to Frogger reaching a home bay cell, i.e. a cell in row 12 whose column is marked as a home bay in the map.

3.1.1 Frog is never alive on a vehicle cell

3.2 Properties Verification

The following properties are expressed in TCTL, a simplified version is used in UPPAAL for the formal verification. Each property is briefly commented to explain its meaning with respect to the system behavior.

Absence of deadlock (without Frogger)

$$\text{(I)}: \quad \forall\Box \neg\, deadlock$$

Verified on the system without the Frogger automaton, this property states that along every path the model never reaches a deadlock state, i.e. the lanes and the diving-turtle cycle can always progress.

All vehicles and floating objects can only move inside their predefined row

$$\text{(II)}: \quad \forall\Box \left(\forall i \ \ L[i].row = i\right)$$

This property states that each vehicle and each floating object remains within its predefined row and never changes lane. It holds by construction, since the lane templates do not include any transition that would allow an object to move vertically.

Nonetheless, it can still be explicitly modelled as "the lane never changes its row" and verified, in order to confirm that no unexpected behaviour causes an object to leave its lane.

Frogger can reach the home bay in position (0,12)–(1,12)

$$\text{(III)}: \quad \exists\Diamond \left(frog_y = 12 \ \wedge \ home_bays[0] = true\right)$$

It's saying that exists at least one path that eventually will lead to Frogger reaching a home bay cell, i.e. a cell in row 12 whose column is marked as a home bay in the map.

Frogger can score 17 points reaching the home bay in position (8,12)–(9,12)

$$\text{(IV)}: \quad \exists\Diamond \left(home_bays[3] = true \ \wedge \ frog_y = 12 \ \wedge \ score = 17\right)$$

A refinement of the previous property: there exists a path on which Frogger reaches the home bay at $(8, 12)$–$(9, 12)$ while holding exactly 17 points.

Frogger can reach a yellow cell in row 6 after the player has lost two lives

$$\text{(V)}: \quad \exists\Diamond \left(frog_y = 6 \ \wedge \ lives = MAX_LIVES - 2\right)$$

There exists a path on which Frogger reaches a cell in row 6 (the safe median) after having lost exactly two lives, i.e. with three of the initial five remaining.

Frogger may stand on a log (with five lives) 10 seconds after the game starts

$$\text{(VI)}: \quad \exists\Diamond \left(lives = MAX_LIVES \wedge LANE_STRUCTURES[frog_y][frog_x] = L \wedge frog_timer = 100 \right)$$

There exists a path on which, within 10 seconds from the start (100 ticks, since one tick is 0.1 s) and still with all five lives, Frogger is standing on a log.

Frogger can die riding a diving turtle that transitions to the submerged state

$$\text{(VII)}: \quad \exists\Diamond \left(diving_turtles_are_submerged \wedge frog_on_diving_turtle \right)$$

There exists a path on which Frogger is standing on a diving-turtle group at the moment it enters the submerged phase, which causes the cell to become open water and the frog to drown. This checks the interaction between the **DivingTurtleCycle** and the **FloatingLane** through the `diving_turtles_submerged` channel.

3.3 Stochastic Verification

4 Conclusion

To summarize, we modelled and formally verified a simplified version of Frogger in UPPAAL, developing both a symbolic and a stochastic version of the system as a network of Timed Automata that captures all the core mechanics of the game. We verified a set of TCTL properties, proving both the correctness of the system and the reachability of requested queries. The work highlighted some design trade-offs adopted to avoid the state space explosion during verification, keeping the final model compact and efficient while preserving a faithful representation of the game logic.